
مهندسی نرم افزار
در عصر
هوش مصنوعی



جلسه اول

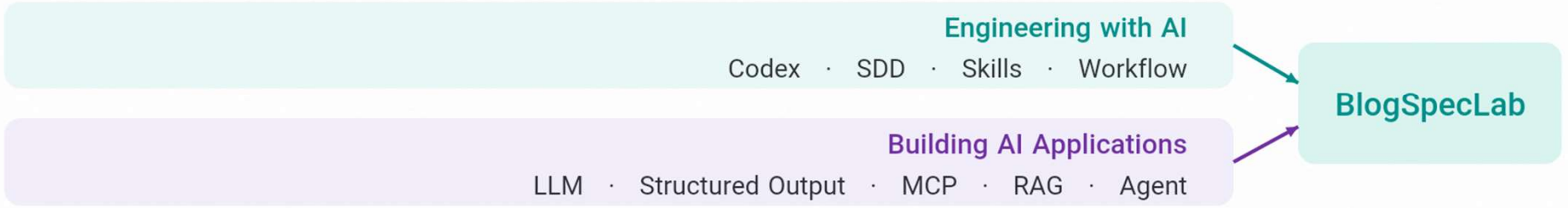
از استفاده از AI تا مهندسی نرم‌افزار با AI



- تغییر نقش هوش مصنوعی در چرخهٔ توسعهٔ نرم‌افزار
- تفاوت تولید کد با تحویل نرم‌افزار قابل‌اعتماد
- حرکت از Promptهای پراکنده به فرایند مهندسی‌شده
- نقش توسعه‌دهنده، معمار و تیم در کنار AI
- دو هدف دوره: توسعه با AI و توسعهٔ محصول مبتنی بر AI
- یادگیری مفاهیم از طریق یک پروژهٔ واقعی

دو مسیر آموزشی در یک دوره

- مسیر اول: استفاده از AI در مهندسی و توسعهٔ نرم‌افزار
- Spec Kit، Codex، SDD و معماری محیط توسعه
- Agent Skills، ابزارهای اختصاصی و Workflow
- مسیر دوم: اضافه کردن قابلیت‌های AI به محصول
- RAG، MCP، Tools، Structured Output، LLM و Agent
- اتصال دو مسیر با پروژهٔ مشترک BlogSpecLab



نقشهٔ آموزشی دوره؛ جلسات ۱ تا ۵

- جلسهٔ ۱: مسئلهٔ محصول، Codex، SDD، Vibe Coding و Spec Kit
- جلسهٔ ۲: معماری محیط توسعه، منابع حقیقت، Skill، ADR، AGENTS.md و CLI
- جلسهٔ ۳: توسعهٔ انتهابه‌انتها، آزمون، Multi-Agent و Workflow
- جلسهٔ ۴: مبانی Context Window، Token، LLM و محدودیت‌های مدل
- جلسهٔ ۵: Prompt Engineering و Context Engineering



نقشهٔ آموزشی دوره؛ جلسات ۶ تا ۱۰

- جلسهٔ ۶: Responses API، Structured Output و اتصال مدل به برنامه
- جلسهٔ ۷: MCP، Tool Calling و دسترسی کنترل‌شده به ابزارها
- جلسهٔ ۸: RAG، Context Engineering و پایگاه برداری Qdrant
- جلسهٔ ۹: طراحی Agent کاربردی در محصول، ابزارها و Human Approval
- جلسهٔ ۱۰: Observability، Security، Evaluation و معماری Production

۶ Structured Output ← ۷ MCP و Tools ← ۸ Qdrant و RAG ← ۹ Agent و تأیید انسانی ← ۱۰ Production

API · LLM · Tools · Vector Database · Agent · Monitoring



BlogSpecLab؛ پروژه‌ای که همراه دوره رشد می‌کند



- شروع با مدیریت پست، نویسنده، ویراستار و فرایند انتشار
- تعریف نیازمندی‌ها و طراحی با SDD و Spec Kit
- پیاده‌سازی و آزمون با کمک Codex و ابزارهای توسعه
- افزودن قابلیت AI Content Review با خروجی ساخت‌یافته
- توسعهٔ قابلیت‌ها با MCP، RAG و Agent
- مشاهدهٔ مسیر تکامل از نیازمندی تا معماری عملیاتی

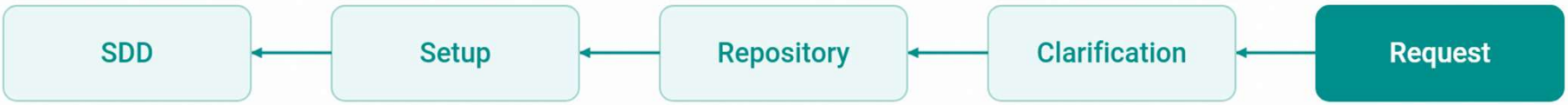
یک درخواست جدید از تیم محصول

- قرار است یک پروژهٔ واقعی را از صفر شروع کنیم
- نقطهٔ شروع ما یک درخواست خام از تیم محصول است
- هنوز Specification دقیق نداریم
- قرار نیست مستقیم وارد کدنویسی شویم
- اول باید ابهام‌ها را ببینیم و مسئله را روشن کنیم



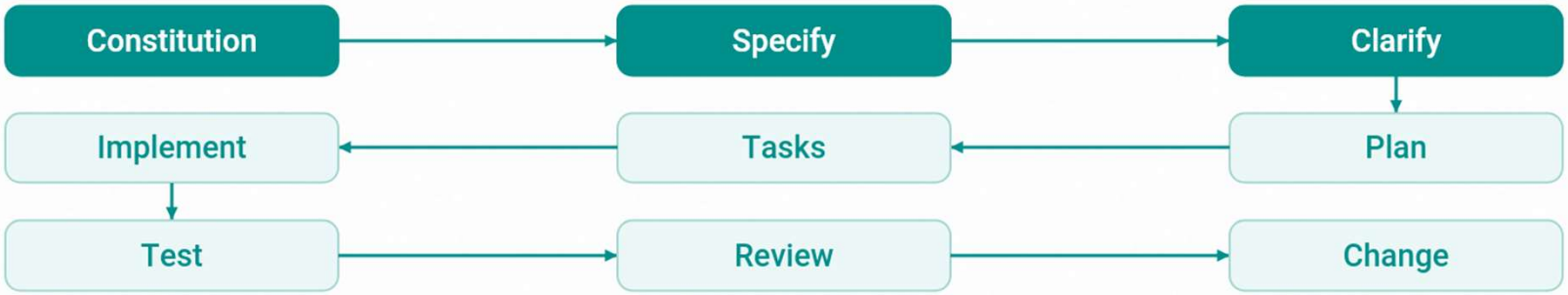
امروز از کجا شروع می‌کنیم؟

- دریافت درخواست خام محصول
- تشخیص ابهام‌ها و نیاز به شفاف‌سازی
- ایجاد یک Repository خالی
- راه‌اندازی Spec Kit در VS Code
- شروع چرخهٔ SDD از روی مسئلهٔ واقعی
- فعلاً هنوز وارد پیاده‌سازی Feature نمی‌شویم



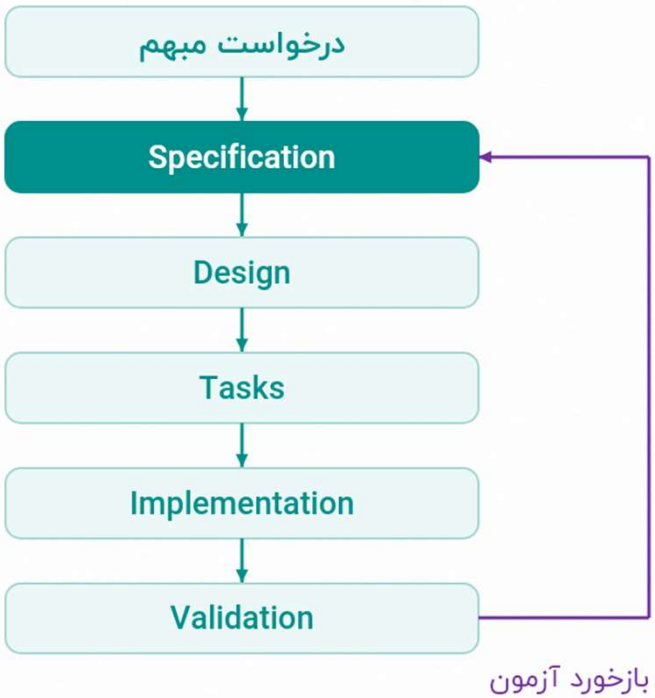
الان در چرخهٔ SDD کجا هستیم؟

- SDD توسعه را از Specification شروع می‌کند
- هر قابلیت باید از یک مسیر مشخص عبور کند
- همهٔ مراحل را یک‌باره جلو نمی‌رویم
- امروز تمرکز ما روی ابتدای چرخه است
- فعلاً با سه گام شروع می‌کنیم: Constitution، Specify و Clarify



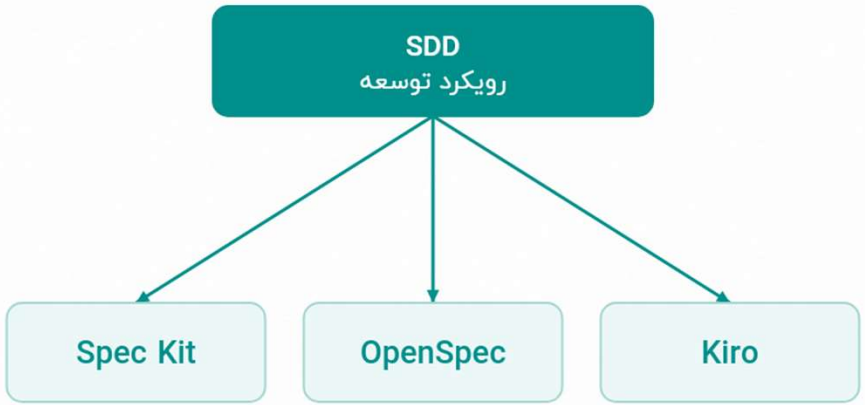
یادآوری SDD؛ چرا از مشخصات شروع می‌کنیم؟

- SDD یا Spec-Driven Development؛ توسعهٔ مبتنی بر مشخصات
- تبدیل درخواست خام به قرارداد قابل فهم و آزمون
- تقدم تعریف رفتار مورد انتظار بر پیاده‌سازی
- Specification به‌عنوان مرجع مشترک انسان و Agent
- چرخهٔ تدریجی: Specification ، سپس Design ، سپس Tasks ، سپس Implementation ، سپس Validation
- مدیریت تغییرات از طریق اصلاح مشخصات، نه فقط کد



چرا Spec Kit؟ آیا جایگزینی دارد؟

- SDD یک رویکرد توسعه است؛ Spec Kit ابزار اجرای آن
- GitHub Spec Kit؛ قالب‌ها، فرایندها و اتصال به Coding Agentها
- OpenSpec؛ مدیریت سبک‌تر مشخصات و تغییرات
- Kiro؛ محیط توسعه با فرایند Spec-driven
- امکان پیاده‌سازی SDD با الگوها و Artifactهای اختصاصی تیم
- انتخاب Spec Kit برای شفافیت مراحل و قابلیت آموزش با Codex



ابزارهای اجرای رویکرد

سه قدم اول ما: Constitution، Specify و Clarify



- Constitution قواعد پایدار و سراسری پروژه را تعریف می‌کند.
- Specify نیازمندی‌های یک قابلیت را بیان می‌کند، نه جزئیات کدنویسی آن.
- Clarify ابهام‌های مؤثر بر طراحی و آزمون را برطرف می‌کند.
- خروجی Clarify باید به Specification برگردد.
- پس از تأیید این مراحل وارد Plan می‌شویم.

گام اول



گام دوم



Constitution و AGENTS.md؛ دو نقش متفاوت

Constitution

چه چیزهایی باید همیشه درست بمانند؟

- اصول پایدار و الزام‌آور پروژه
- قواعد معماری و جهت وابستگی‌ها
- مرزهای امنیت و مسئولیت
- قواعد کیفیت و حاکمیت تغییر
- مرجع کنترل Spec، Plan و Implementation

AGENTS.md

Agent در این Repository چطور باید کار کند؟

- از کجا Context پروژه را پیدا کند
- قبل از تغییر چه فایل‌هایی را بخواند
- چه Workflowای را دنبال کند
- چه کارهایی را بدون تأیید انجام ندهد
- چگونه Test، Build و Validation را اجرا کند

Constitution = قواعد حاکم بر پروژه

AGENTS.md = راهنمای عملیاتی Coding Agent

Codex



گام سوم



Specify؛ از درخواست محصول تا رفتار قابل انتظار

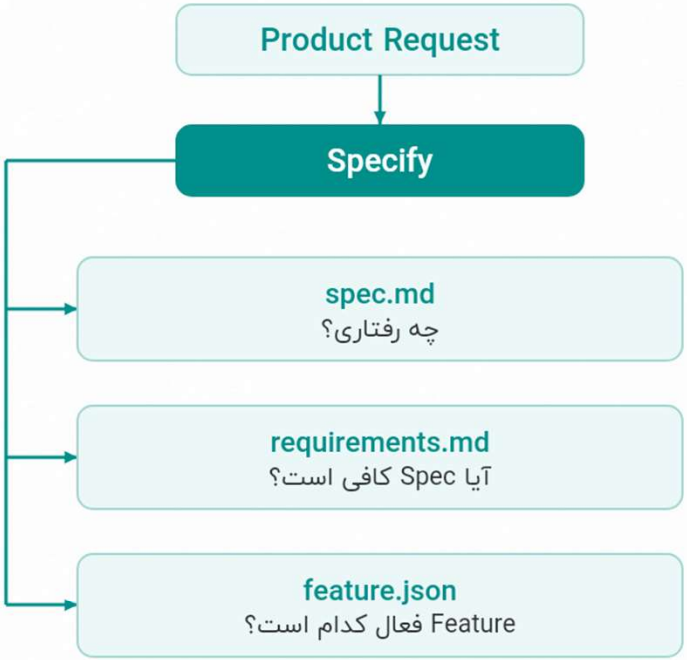


- ورودی Specify یک نیاز یا درخواست محصول است
- Specification توضیح می‌دهد سیستم چه رفتاری باید داشته باشد
- تمرکز روی «What» و «Why»، نه «How»
- سناریوهای اصلی و معیارهای پذیرش باید قابل بررسی باشند
- تصمیم‌های فنی زودهنگام نباید وارد Specification شوند
- ابهام‌های مهم باید برای Clarify قابل مشاهده باقی بمانند

Specification = قرارداد رفتار مورد انتظار، نه طراحی پیاده‌سازی

گام چهارم

خروجی Specify چه بود؟



- spec.md : تعریف رفتار و نیازمندی‌های Feature
- User Storyها، Acceptance Scenarioها و Functional Requirementها
- Edge Case، Success Criteriaها و Assumptionها
- requirements.md : کنترل کیفیت Specification
- .specify/feature.json : مشخص کردن Feature فعال
- خروجی Specify هنوز طراحی فنی یا کد نیست

Clarify؛ پیدا کردن تصمیم‌های پنهان در Specification



- Specify درخواست را به Specification تبدیل کرد
- Clarify به‌دنبال ابهام‌ها و تصمیم‌های حل‌نشده می‌گردد
- تمرکز روی مواردی است که چند تفسیر معتبر دارند
- State، Permission، Role، Scope و Failureها بررسی می‌شوند
- پاسخ Clarification باید داخل خود Specification ثبت شود
- بعد از Clarify، Spec باید برای Plan قابل اتکاتر باشد

Specify می‌پرسد: «چه چیزی می‌خواهیم؟»
Clarify می‌پرسد: «آیا واقعاً منظورمان از آن روشن است؟»

گام پنجم

Plan؛ از «چه می‌خواهیم؟» به «چطور می‌سازیم؟»



- Specification رفتار مورد انتظار را مشخص کرد
- Clarify ابهام‌های مهم را برطرف کرد
- Plan راه‌حل فنی را برای همان Specification طراحی می‌کند
- اینجا فناوری، معماری و ساختار پروژه وارد تصمیم می‌شوند
- Plan باید با Constitution سازگار باشد
- هنوز وارد تقسیم کار اجرایی و کدنویسی نمی‌شویم

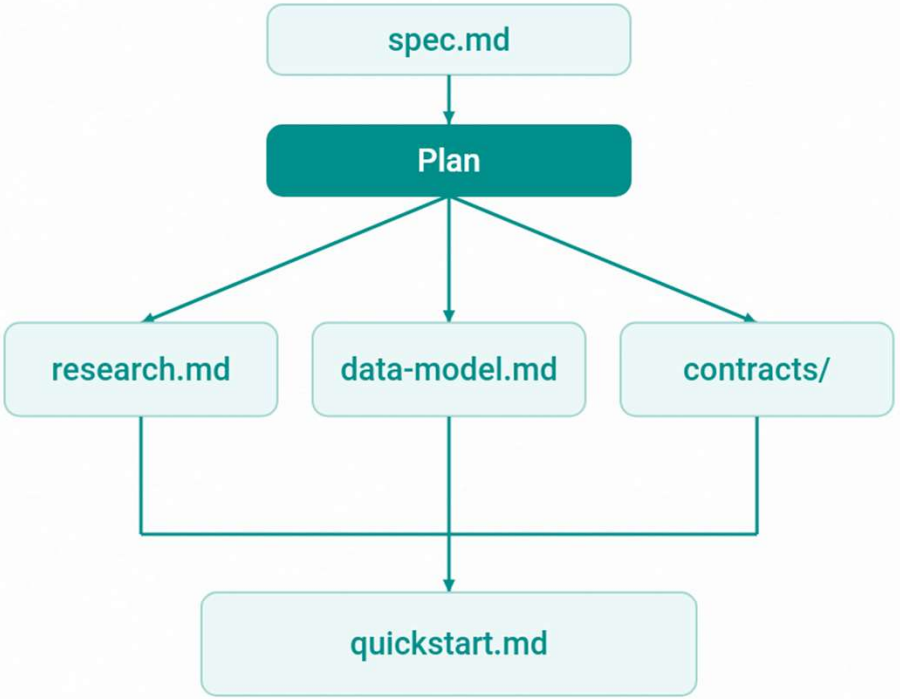
Spec = What & Why

Plan = How

گام ششم

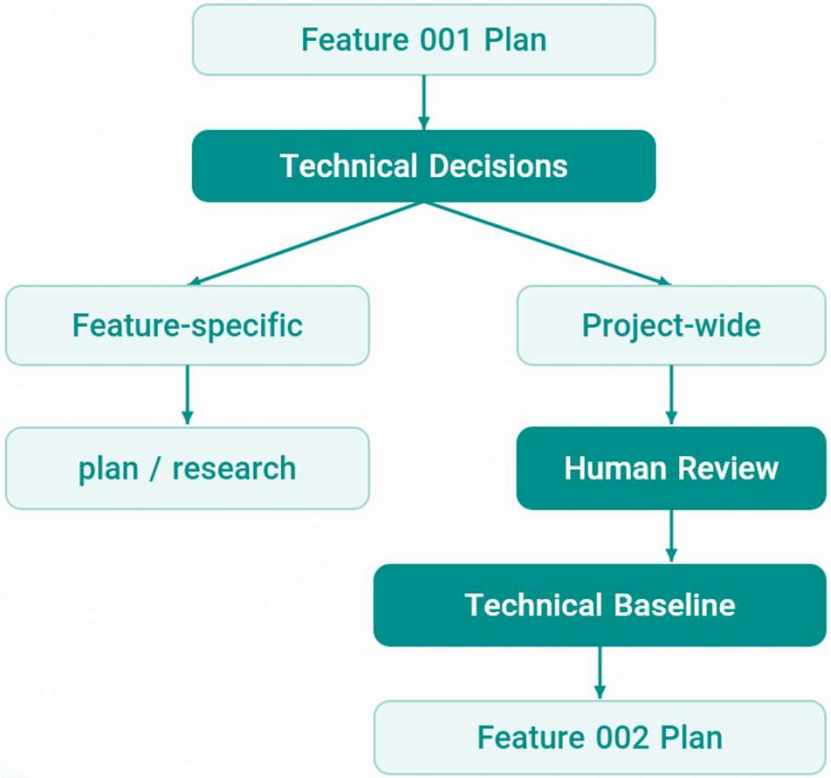


خروجی مرحلهٔ Plan چیست؟



- plan.md : طراحی فنی کلی Feature
- research.md : تصمیم‌های فنی، دلیل انتخاب و گزینه‌های جایگزین
- data-model.md : موجودیت‌ها، روابط و State Transitionها
- contracts/ : قراردادهای خارجی سیستم
- quickstart.md : مسیر اعتبارسنجی End-to-End
- این مرحله هنوز Implementation یا Tasks نیست

چرا بعد از Plan یک Technical Baseline می‌سازیم؟



- بعضی تصمیم‌های Plan فقط مخصوص همان Feature هستند
- بعضی تصمیم‌ها برای کل Repository معتبر می‌شوند
- تصمیم سراسری نباید در Plan یک Feature دفن شود
- Featureهای بعدی نباید همان تصمیم را دوباره بگیرند
- Technical Baseline مرجع تصمیم‌های فنی تثبیت‌شده است
- AGENTS.md Agent را به این مرجع هدایت می‌کند

Feature Plan کشف می‌کند؛ Technical Baseline تثبیت می‌کند.

گام هفتم



Tasks؛ تبدیل طراحی به واحدهای قابل اجرا



- Plan مشخص کرد چطور Feature را می‌سازیم
- Tasks این طراحی را به گام‌های اجرایی کوچک تبدیل می‌کند
- Taskها باید ترتیب وابستگی مشخص داشته باشند
- هر Task باید محدوده و فایل هدف روشن داشته باشد
- Taskهای Feature براساس User Storyها گروه‌بندی می‌شوند
- خروجی باید مستقیماً برای Implementation قابل استفاده باشد

Plan = Design

Tasks = Executable Work Breakdown

گام هشتم

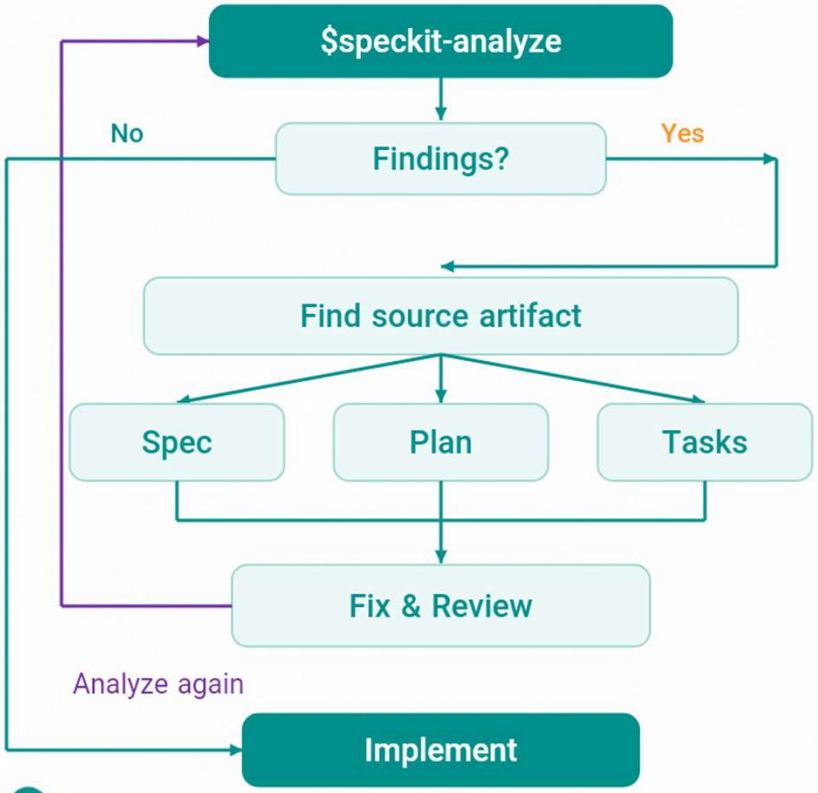
Analyze؛ قبل از کدنویسی تناقض‌ها را پیدا کنیم



- Spec می‌گوید چه چیزی باید ساخته شود
- Plan می‌گوید چگونه قرار است ساخته شود
- Tasks مشخص می‌کند چه کارهایی باید انجام شوند
- Analyze این سه Artifact را با هم تطبیق می‌دهد
- Coverage، تناقض، ابهام و نقض Constitution بررسی می‌شوند
- Analyze هیچ فایلی را تغییر نمی‌دهد

Analyze = Consistency Check قبل از هزینهٔ Implementation

اگر Analyze مشکل پیدا کرد چه می‌کنیم؟



- Analyze فقط گزارش می‌دهد؛ فایل‌ها را تغییر نمی‌دهد
- هر Finding باید در Artifact منبع خودش اصلاح شود
- Requirement مشکل دارد : spec.md
- Design مشکل دارد : plan.md / Design Artifacts
- Task ناقص است : tasks.md
- پس از اصلاح، Analyze دوباره اجرا می‌شود
- اگر Finding مهمی باقی نماند، وارد Implementation می‌شویم

Fix the source, then analyze again.

گام دهم

قبل از Feature، مسیر قابل اعتماد را می‌سازیم



- هنوز وارد منطق مدیریت پست نشده‌ایم
- ابتدا هویت کاربر را از مجوز جاری جدا می‌کنیم
- Application مرجع کنترل Permissionهای Use Case است
- عملیات تغییردهنده باید مرز تراکنش مشخص داشته باشند
- Infrastructure فقط پیاده‌سازی قراردادهای Core است
- خطاهای Application مستقل از HTTP تعریف می‌شوند

Authentication می‌گوید «چه کسی هستی؟»
Authorization می‌گوید «الان اجازهٔ چه کاری داری؟»

گام یازدهم



اولین Increment واقعی محصول: User Story 1



- زیرساخت پایه آماده است؛ حالا وارد رفتار کسب‌وکار می‌شویم
- US1: نویسنده پست خودش را مدیریت می‌کند
- ابتدا تست‌های Domain، Application و HTTP نوشته می‌شوند
- سپس Application Service، Persistence، Aggregate و Endpoint ساخته می‌شوند
- مالکیت و State Transitionها باید در مرز درست اعمال شوند
- پایان این فاز یک MVP مستقل برای نویسنده است

برای اولین بار از زیرساخت وارد رفتار واقعی محصول می‌شویم.

گام دوازدهم



User Story 2؛ از تولید محتوا تا انتشار کنترل‌شده



US1 فقط مسیر نویسنده را کامل کرد

US2 مسیر Approval، Request Changes، Review و Publish را اضافه می‌کند

تأیید به محتوای مشخص پست وابسته است

خودتأییدی ممنوع است

Permission فعلیِ Review و Publish مستقل بررسی می‌شود

تصمیم Review و تغییر وضعیت باید اتمی ثبت شوند

Approval یک برچسب روی Post نیست؛ تصمیمی دربارهٔ یک نسخهٔ مشخص از .محتواست

گام سیزدهم



از مسیر درست تا سیستم مقاوم



مسیر Publish → Approval → Review → Author کامل شده

حالا باید ثابت کنیم مسیرهای غیرمجاز واقعاً بسته‌اند

Public Read نباید دادهٔ منتشرنشده را افشا کند

Permission باید در لحظهٔ عملیات دوباره بررسی شود

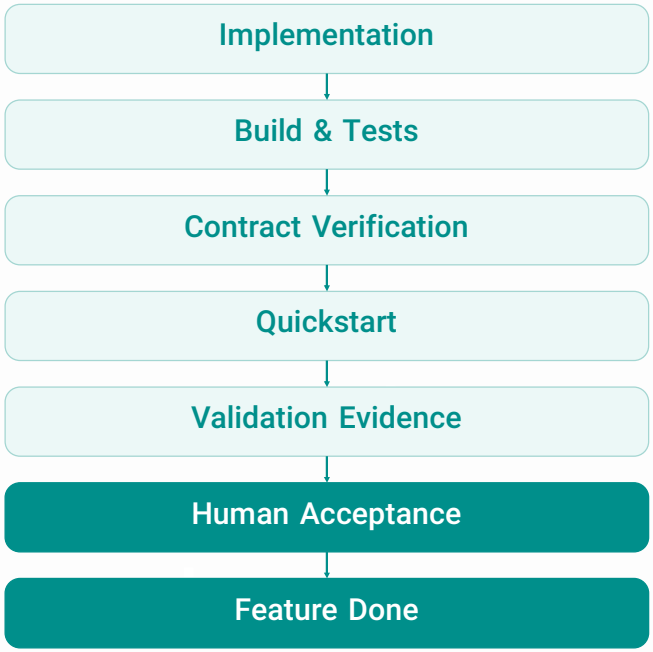
درخواست‌های هم‌زمان نباید State ناسازگار ایجاد کنند

Publish تکراری نباید انتشار جدید بسازد

US3 = اثبات اینکه سیستم فقط در Happy Path درست کار نمی‌کند

گام چهاردهم

از پیاده‌سازی به اثبات تکمیل



تمام User Storyها و مسیرهای امنیتی پیاده‌سازی شده‌اند

Build و Test سبز بودن، پایان کار نیست

قرارداد API باید با Implementation تطبیق داده شود

Quickstart باید مسیر واقعی اجرا و Demo را ثبت کند

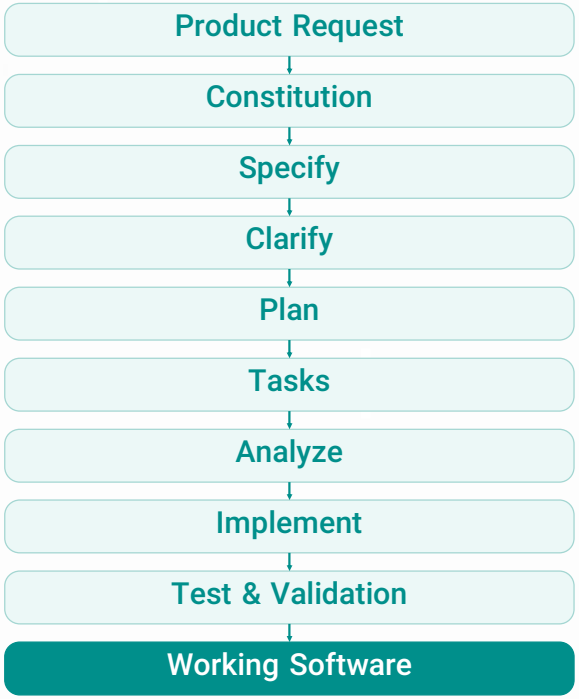
شواهد تحقق Success Criteria باید مستند شوند

پذیرش انسانی جدا از آزمون خودکار است

Implementation Complete ≠ Feature Accepted

گام پانزدهم

تا اینجا چه ساختیم؟



No AI yet

یک درخواست خام محصول را به Specification قابل آزمون تبدیل کردیم

ابهام‌ها را با Clarify به تصمیم‌های صریح تبدیل کردیم

معماری و تصمیم‌های فنی را در Plan طراحی کردیم

Tasks مسیر Implementation را به واحدهای قابل کنترل شکست

Analyze سازگاری Plan، Spec و Tasks را قبل از کدنویسی بررسی کرد

نرم‌افزار پایه بدون وابستگی به AI ساخته، آزموده و اعتبارسنجی شد

AI را به یک نرم‌افزار دارای قرارداد اضافه می‌کنیم؛ نه اینکه قرارداد نرم‌افزار را به AI واگذار .کنیم